PYTHON ACTIVITY-BASED LEARNING

10 Complete Case Study Solutions

B.Tech | Python Programming

Student Name	_____
Roll Number	_____
Branch / Section	_____
Faculty / Lab Mentor	_____
Academic Year	_____

Prepared according to the uploaded college circular. Each case study includes a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output, and conclusion.

Important: Replace the sample student details and verify the solutions with your faculty's exact rules before submission. The sample thresholds, tariffs, rates, and policies are for programming practice.

Contents

The following 10 case studies follow the first ten case-study titles in the supplied assignment list.

1. University Scholarship Allocation

2. Progressive Electricity Billing

3. Hospital Billing and Patient Classification

4. Employee Payroll and Deduction Analysis

5. Multi-Plan Telecommunications Cost Optimization

6. Railway Fare and Reservation Calculator

7. Retail Billing with Multiple Discount Rules

8. Manufacturing Cost and Profitability Analysis

9. Multi-Currency Travel Budget Analysis

10. Campus Event Financial Planning

Required deliverables checklist

■ Problem statement

■ Objectives

■ Assumptions

■ Algorithm

■ Flowchart

■ Pseudocode

■ Python source code

■ Test cases

■ Sample output

■ Conclusion

CASE STUDY 1: University Scholarship Allocation

Reference in supplied list: Case Study 1

1. Problem Statement

Design a simple scholarship screening program that checks each applicant's aggregate marks, subject-wise minimum scores, attendance, and family income. The program should identify eligible applicants and rank them using aggregate marks, with attendance as the tie-breaker.

2. Objectives

• Apply conditional statements to multiple eligibility requirements.

• Store applicant records in lists and nested lists.

• Rank eligible applicants using clear, deterministic rules.

3. Assumptions

• All marks and attendance are percentages from 0 to 100.

• An applicant must have aggregate marks of at least 80, every listed subject mark of at least 70, attendance of at least 75%, and annual family income no greater than ■250,000.

• The sample contains four applicants; the thresholds are illustrative and should be changed if faculty provides different rules.

• Ties are resolved by higher attendance; if both are equal, the earlier applicant in the input list is shown first.

4. Algorithm

1. Start and define the eligibility thresholds.

2. Read each applicant's name, aggregate, subject marks, attendance, and income.

3. Check every criterion and record the reason if the applicant is not eligible.

4. Collect eligible applicants in a list.

5. Sort eligible applicants by aggregate marks and then attendance, both in descending order.

6. Display the ranked list and end.

5. Flowchart

```
START\n  ↓\nSet scholarship criteria\n  ↓\nRead applicant records\n  ↓\nCheck marks, subject minimums, atte
```

6. Pseudocode

```
SET eligible = empty list\nFOR each applicant\n    CHECK aggregate, each subject mark, attendance, income\n    IF every condi
```

7. Python Source Code

```
# University Scholarship Allocation
aggregate_min = 80
subject_min = 70
attendance_min = 75
income_max = 250000

applicants = [
    ["Asha", 88, [82, 85, 91], 90, 180000],
    ["Bharat", 79, [80, 76, 84], 92, 150000],
    ["Charan", 91, [88, 69, 95], 86, 120000],
    ["Divya", 91, [90, 92, 89], 86, 200000]
]

eligible = []
rejected = []

for applicant in applicants:
    name, aggregate, subjects, attendance, income = applicant
    reasons = []
    if aggregate < aggregate_min:
```

```
            reasons.append("aggregate below minimum")
        for mark in subjects:
            if mark < subject_min:
                reasons.append("subject mark below minimum")
                break
        if attendance < attendance_min:
            reasons.append("attendance below minimum")
        if income > income_max:
            reasons.append("income above limit")

        if len(reasons) == 0:
            eligible.append(applicant)
        else:
            rejected.append([name, reasons])

eligible.sort(key=lambda item: (item[1], item[3]), reverse=True)

print("ELIGIBLE SCHOLARSHIP APPLICANTS")
rank = 1
for applicant in eligible:
    print(rank, applicant[0], "Aggregate:", applicant[1],
          "Attendance:", applicant[3])
    rank += 1

print("\nNOT ELIGIBLE")
for item in rejected:
    print(item[0], "-", ", ".join(item[1]))
```

8. Test Cases

Input / condition	Expected result
Asha: all criteria pass	Eligible; ranked by aggregate
Bharat: aggregate 79	Rejected: aggregate below minimum
Charan: one subject mark 69	Rejected: subject mark below minimum
Divya: all criteria pass	Eligible; ties handled by attendance

9. Sample Output

```
ELIGIBLE SCHOLARSHIP APPLICANTS\n1 Divya Aggregate: 91 Attendance: 86\n2 Asha Aggregate: 88 Attendance: 90\n\nNOT ELIGIBLE\nB
```

10. Conclusion

The program applies multiple eligibility conditions, explains failed criteria, and ranks eligible applicants with a defined tie-breaking rule.

CASE STUDY 2: Progressive Electricity Billing

Reference in supplied list: Case Study 2

1. Problem Statement

Calculate an electricity bill using progressive consumption slabs, a fixed charge, a surcharge, and a rebate. Compare the current bill with the previous month's bill and flag a large increase.

2. Objectives

• Use if/elif conditions to calculate progressive slab charges.

• Apply fixed charges, surcharge, and rebate in order.

• Compare bills across months and identify unusual increases.

3. Assumptions

• First 100 units cost ■2 per unit; next 100 units cost ■3 per unit; units above 200 cost ■5 per unit.

• Fixed charge is ■50. A 5% surcharge applies to energy plus fixed charge.

• If consumption is 100 units or less, a ■20 rebate is applied after surcharge.

• An increase greater than 30% over the previous bill is flagged.

4. Algorithm

1. Read previous and current month consumption.

2. Calculate energy charge by applying each slab only to units within that slab.

3. Add fixed charge and calculate surcharge.

4. Apply the rebate if its condition is met; do not allow a negative final bill.

5. Calculate percentage change and flag an increase greater than 30%.

6. Display itemized charges and comparison.

5. Flowchart

```
START → Read previous/current units\n  ↓\nCalculate slab-wise energy charge\n  ↓\nAdd fixed charge and sur
```

6. Pseudocode

```
FUNCTION bill(units)\n    CALCULATE charge for first 100 units\n    CALCULATE charge for next 100 units\n    CALCULATE charge
```

7. Python Source Code

```
# Progressive Electricity Billing
def calculate_bill(units):
    if units < 0:
        return None

    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    fixed = 50
    subtotal = energy + fixed
    surcharge = subtotal * 0.05
    total = subtotal + surcharge
    rebate = 20 if units <= 100 else 0
    total = max(0, total - rebate)
    return energy, fixed, surcharge, rebate, total

previous_units = 120
current_units = 260
previous = calculate_bill(previous_units)
current = calculate_bill(current_units)
```

```
if previous is None or current is None:
    print("Invalid units entered")
else:
    print("Previous bill: Rs.", round(previous[4], 2))
    print("Current bill: Rs.", round(current[4], 2))
    print("Energy charge: Rs.", round(current[0], 2))
    print("Fixed charge: Rs.", current[1])
    print("Surcharge: Rs.", round(current[2], 2))
    print("Rebate: Rs.", current[3])
    increase = ((current[4] - previous[4]) / previous[4]) * 100
    print("Increase: ", round(increase, 2), "%")
    if increase > 30:
        print("Alert: unusual increase in bill")
    else:
        print("No unusual increase")
```

8. Test Cases

Units	Expected energy charge
0	■0
100	■200
150	■350
250	■850
-5	Invalid units

9. Sample Output

```
Previous bill: Rs. 325.5
Current bill: Rs. 892.5
Energy charge: Rs. 800
Fixed charge: Rs. 50
Surcharge: Rs. 42.5
Rebate: Rs. 0
Increase: 174.19 %
Alert: unusual increase in bill
```

10. Conclusion

The solution calculates progressive charges correctly and uses month-to-month comparison to flag a substantial bill increase.

CASE STUDY 3: Hospital Billing and Patient Classification

Reference in supplied list: Case Study 3

1. Problem Statement

Prepare a hospital bill from consultation, diagnostic, and treatment charges. Apply a concession when the supplied policy conditions are met, validate the bill, and classify the patient's payment status.

2. Objectives

• Calculate a total from several service charges.

• Apply a conditional concession and validate inputs.

• Classify the bill based on payment against the final amount.

3. Assumptions

• A 10% concession applies if the patient is a senior citizen (age 60 or above) or has a valid concession card.

• The concession applies to the sum of consultation, diagnostic, and treatment charges.

• Payment status is 'Paid' when payment is at least the final bill, 'Partially Paid' when payment is greater than zero but less than the bill, and 'Unpaid' when payment is zero.

• All amounts and age must be non-negative.

4. Algorithm

1. Read patient name, age, charges, concession-card status, and amount paid.

2. Validate age and all charges/payment are non-negative.

3. Add consultation, diagnostic, and treatment charges.

4. Apply a 10% concession if either eligibility condition is true.

5. Compare payment with final bill and classify payment status.

6. Display itemized bill and status.

5. Flowchart

```
START → Read patient and charge details\n  ↓\nValidate non-negative values\n  ↓\nAdd service charges\n  ↓\
```

6. Pseudocode

```
READ patient details and charges\nIF any value is negative: DISPLAY invalid input\nELSE\n    total = consultation + diagnostic
```

7. Python Source Code

```
# Hospital Billing and Patient Classification
name = "Ravi"
age = 65
consultation = 500
diagnostics = 1200
treatment = 2500
has_card = False
paid = 3000

values = [age, consultation, diagnostics, treatment, paid]
if min(values) < 0:
    print("Invalid input: values cannot be negative")
else:
    subtotal = consultation + diagnostics + treatment
    discount = 0
    if age >= 60 or has_card:
        discount = subtotal * 0.10

    final_bill = subtotal - discount
    balance = final_bill - paid

    if paid >= final_bill:
        status = "Paid"
```

```
elif paid > 0:
    status = "Partially Paid"
else:
    status = "Unpaid"

print("Patient:", name)
print("Subtotal: Rs.", round(subtotal, 2))
print("Concession: Rs.", round(discount, 2))
print("Final bill: Rs.", round(final_bill, 2))
print("Amount paid: Rs.", round(paid, 2))
print("Balance due: Rs.", round(max(0, balance), 2))
print("Payment status:", status)
```

8. Test Cases

Scenario	Expected result
Age 65, subtotal ■4,200	10% concession = ■420
Payment ■3,000, final bill ■3,780	Partially Paid; balance ■780
No concession and payment 0	Unpaid
Negative diagnostic charge	Invalid input

9. Sample Output

```
Patient: Ravi\nSubtotal: Rs. 4200\nConcession: Rs. 420.0\nFinal bill: Rs. 3780.0\nAmount paid: Rs. 3000\nBalance due: Rs. 780
```

10. Conclusion

The program creates a transparent invoice, applies policy-based concessions, rejects invalid values, and reports the payment balance.

CASE STUDY 4: Employee Payroll and Deduction Analysis

Reference in supplied list: Case Study 4

1. Problem Statement

Calculate an employee's gross salary from basic pay, allowances, overtime, and incentives. Apply progressive deductions and print a formatted salary statement.

2. Objectives

• Compute gross salary from multiple components.

• Use progressive deduction slabs with conditional logic.

• Generate a clear salary statement.

3. Assumptions

• HRA is 20% of basic pay and other allowance is 10% of basic pay.

• Overtime is paid at ■200 per hour; incentive is supplied as a fixed amount.

• Illustrative deduction rule: 0% on the first ■20,000 of gross salary, 5% on the next ■20,000, and 10% on any amount above ■40,000.

• These are assignment example rules, not official tax advice.

4. Algorithm

1. Read basic pay, overtime hours, and incentive.

2. Validate that all values are non-negative.

3. Calculate allowances, overtime pay, and gross salary.

4. Calculate deductions progressively across the three ranges.

5. Subtract deductions from gross salary to obtain net salary.

6. Display each component in a salary statement.

5. Flowchart

```
START → Read pay inputs\n  ↓\nValidate inputs\n  ↓\nCalculate HRA, allowance, overtime\n  ↓\nGross salary
```

6. Pseudocode

```
READ basic pay, overtime hours, incentive\nHRA = 20% of basic\nOTHER = 10% of basic\nOVERTIME = hours * rate\nGROSS = basic +
```

7. Python Source Code

```
# Employee Payroll and Deduction Analysis
basic = 30000
overtime_hours = 6
incentive = 2500

if min(basic, overtime_hours, incentive) < 0:
    print("Invalid salary input")
else:
    hra = basic * 0.20
    other_allowance = basic * 0.10
    overtime_pay = overtime_hours * 200
    gross = basic + hra + other_allowance + overtime_pay + incentive

    if gross <= 20000:
        deduction = 0
    elif gross <= 40000:
        deduction = (gross - 20000) * 0.05
    else:
        deduction = 20000 * 0.05 + (gross - 40000) * 0.10

    net = gross - deduction
    print("SALARY STATEMENT")
    print("Basic pay: Rs.", round(basic, 2))
```

```
print("HRA: Rs.", round(hra, 2))
print("Other allowance: Rs.", round(other_allowance, 2))
print("Overtime pay: Rs.", round(overtime_pay, 2))
print("Incentive: Rs.", round(incentive, 2))
print("Gross salary: Rs.", round(gross, 2))
print("Deduction: Rs.", round(deduction, 2))
print("Net salary: Rs.", round(net, 2))
```

8. Test Cases

Gross salary	Expected deduction
■18,000	■0
■30,000	■500
■50,000	■2,000
Any negative input	Invalid salary input

9. Sample Output

```
SALARY STATEMENT
Basic pay: Rs. 30000
HRA: Rs. 6000.0
Other allowance: Rs. 3000.0
Overtime pay: Rs. 1200
Incentive: Rs. 2500
Gross salary: Rs. 42700.0
Deduction: Rs. 1270.0
Net salary: Rs. 41430.0
```

10. Conclusion

The program combines salary components, calculates deductions by progressive slabs, and presents gross and net pay in a readable format.

CASE STUDY 5: Multi-Plan Telecommunications Cost Optimization

Reference in supplied list: Case Study 5

1. Problem Statement

Compare mobile plans for a user's monthly call/data usage, plan validity, and budget. Calculate extra usage charges and select the least expensive plan that satisfies the user's requirements.

2. Objectives

• Represent plan information using a nested list.

• Calculate excess usage charges for each plan.

• Filter plans by validity and budget, then identify the cheapest eligible option.

3. Assumptions

• Usage and prices are illustrative.

• Each plan contains name, base price, included data in GB, included call minutes, and validity in days.

• Extra data costs ■20 per GB and extra calling costs ■0.50 per minute.

• The plan must have validity of at least 28 days and total cost no greater than the user's budget.

4. Algorithm

1. Read expected data, call minutes, and budget.

2. Store plans in a nested list.

3. For each plan, calculate data and call excess.

4. Add excess charges to base price.

5. Keep plans that meet validity and budget requirements.

6. Choose the eligible plan with the lowest total cost; report if none qualify.

5. Flowchart

```
START → Read usage and budget\n  ↓\nLoad plan list\n  ↓\nFor each plan: calculate excess data/calls\n  ↓\n
```

6. Pseudocode

```
READ data_used, minutes_used, budget\nFOR each plan\n    data_extra = max(0, data_used - included_data)\n    call_extra = max
```

7. Python Source Code

```
# Multi-Plan Telecommunications Cost Optimization
data_used = 18
minutes_used = 450
budget = 500

# Each plan: name, price, included GB, included minutes, validity days
plans = [
    ["Basic", 249, 10, 300, 28],
    ["Value", 399, 20, 500, 56],
    ["Plus", 449, 25, 600, 28],
    ["Short", 199, 15, 400, 14]
]

eligible = []
for plan in plans:
    name, price, data_allowance, call_allowance, validity = plan
    extra_data = max(0, data_used - data_allowance)
    extra_calls = max(0, minutes_used - call_allowance)
    total = price + extra_data * 20 + extra_calls * 0.50

    print(name, "- estimated cost: Rs.", round(total, 2))
    if validity >= 28 and total <= budget:
```

```
        eligible.append([name, total, validity])

if len(eligible) == 0:
    print("No plan meets the validity and budget requirements")
else:
    eligible.sort(key=lambda item: item[1])
    best = eligible[0]
    print("Recommended plan:", best[0])
    print("Estimated total cost: Rs.", round(best[1], 2))
    print("Validity:", best[2], "days")
```

8. Test Cases

Scenario	Expected result
18 GB and 450 minutes	Calculate excess by plan
Budget ■500	Exclude any plan costing above ■500
Plan validity 14 days	Reject for minimum validity 28 days
No eligible plan	Display no suitable plan

9. Sample Output

```
Basic - estimated cost: Rs. 484.0
Value - estimated cost: Rs. 399.0
Plus - estimated cost: Rs. 449.0
Short - estimated cost: Rs. 284.0
Recommended plan: Value
Estimated total cost: Rs. 399.0
Validity: 56 days
```

10. Conclusion

The solution estimates the total cost of each plan, enforces budget and validity constraints, and recommends the lowest-cost eligible plan.

CASE STUDY 6: Railway Fare and Reservation Calculator

Reference in supplied list: Case Study 6

1. Problem Statement

Calculate passenger fares using distance slabs, travel class, age-based concessions, reservation fees, and a cancellation penalty.

2. Objectives

• Use conditions to calculate distance-based fare.

• Apply class multipliers and an age-based concession.

• Calculate a cancellation refund using a defined penalty.

3. Assumptions

• Base fare is ■1.00 per km for the first 100 km, ■0.80 per km for the next 200 km, and ■0.60 per km beyond 300 km.

• Class multipliers are Sleeper 1.0, AC3 1.8, and AC2 2.5.

• Children aged 5–11 receive 50% fare concession; passengers aged 60 or above receive 25% concession. The larger of the two applicable concessions is used.

• Reservation fee is ■40. Cancellation penalty is 20% of fare plus reservation fee; refund cannot be negative.

4. Algorithm

1. Read distance, class, age, and cancellation choice.

2. Validate distance and age.

3. Calculate the distance-based base fare.

4. Multiply by travel-class factor and apply age concession if eligible.

5. Add reservation fee.

6. If cancelled, calculate penalty and refund; otherwise show payable fare.

5. Flowchart

```
START → Read journey details\n  ↓\nValidate distance and age\n  ↓\nCalculate distance slab fare\n  ↓\nAppl
```

6. Pseudocode

```
READ distance, class, age, cancelled\nCALCULATE base fare using progressive distance slabs\nfare = base fare * class multipli
```

7. Python Source Code

```
# Railway Fare and Reservation Calculator
distance = 350
travel_class = "AC3"
age = 65
cancelled = False

class_rates = [["Sleeper", 1.0], ["AC3", 1.8], ["AC2", 2.5]]
class_factor = 0
for item in class_rates:
    if item[0] == travel_class:
        class_factor = item[1]

if distance <= 0 or age < 0 or class_factor == 0:
    print("Invalid journey details")
else:
    if distance <= 100:
        base = distance * 1.00
    elif distance <= 300:
        base = 100 * 1.00 + (distance - 100) * 0.80
    else:
        base = 100 * 1.00 + 200 * 0.80 + (distance - 300) * 0.60

    fare = base * class_factor
```

```
concession = 0
if 5 <= age <= 11:
    concession = 0.50
elif age >= 60:
    concession = 0.25

fare = fare * (1 - concession)
reservation = 40
amount = fare + reservation

print("Base fare: Rs.", round(base, 2))
print("Class:", travel_class, "Multiplier:", class_factor)
print("Age concession:", int(concession * 100), "%")
print("Fare with reservation: Rs.", round(amount, 2))

if cancelled:
    penalty = amount * 0.20
    print("Cancellation penalty: Rs.", round(penalty, 2))
    print("Refund: Rs.", round(max(0, amount - penalty), 2))
else:
    print("Fare payable: Rs.", round(amount, 2))
```

8. Test Cases

Scenario	Expected result
Distance 100 km	Base fare ■100
Distance 350 km	Base fare ■330
Age 65	25% concession
Unknown class	Invalid journey details
Cancellation selected	Penalty and refund shown

9. Sample Output

```
Base fare: Rs. 330.0\nClass: AC3 Multiplier: 1.8\nAge concession: 25 %\nFare with reservation: Rs. 485.5\nFare payable: Rs. 4
```

10. Conclusion

The program combines distance slabs, class multipliers, concessions, and reservation charges, and can also calculate a cancellation refund.

CASE STUDY 7: Retail Billing with Multiple Discount Rules

Reference in supplied list: Case Study 7

1. Problem Statement

Create an itemized retail invoice for multiple products and quantities. Apply category-specific taxes, a threshold discount, and a maximum concession limit.

2. Objectives

• Use lists to represent products and quantities.

• Calculate line totals, tax, and conditional discount.

• Apply a maximum discount cap and produce an itemized invoice.

3. Assumptions

• Each product record stores name, unit price, quantity, and tax percentage.

• A 10% discount applies when the pre-tax subtotal is at least ■2,000.

• The discount is capped at ■500.

• Tax is calculated on the discounted subtotal, using each item's tax rate proportionally for simplicity; actual tax rules may differ.

4. Algorithm

1. Read product records.

2. Validate price, quantity, and tax rate.

3. Calculate each line subtotal and accumulate subtotal and weighted tax.

4. If subtotal reaches ■2,000, calculate 10% discount limited to ■500.

5. Allocate the discount proportionally across taxable lines for calculation.

6. Calculate tax on discounted line amounts and print invoice total.

5. Flowchart

```
START → Read product list\n  ↓\nValidate item values\n  ↓\nCalculate line totals and subtotal\n  ↓\nSubtot
```

6. Pseudocode

```
subtotal = 0\nFOR each product: line_total = price * quantity; add to subtotal\nIF subtotal >= threshold: discount = min(subt
```

7. Python Source Code

```
# Retail Billing with Multiple Discount Rules
products = [
    ["Notebook", 80, 5, 5],
    ["Backpack", 900, 2, 12],
    ["Pen Pack", 120, 3, 5]
]

subtotal = 0
for product in products:
    name, price, quantity, tax_rate = product
    if price < 0 or quantity < 0 or tax_rate < 0:
        print("Invalid product record:", name)
        break
    subtotal += price * quantity
else:
    if subtotal >= 2000:
        discount = min(subtotal * 0.10, 500)
    else:
        discount = 0

    total_tax = 0
    print("ITEMIZED INVOICE")
    for product in products:
```

```
        name, price, quantity, tax_rate = product
        line = price * quantity
        line_discount = 0
        if subtotal > 0:
            line_discount = discount * line / subtotal
        taxable = line - line_discount
        tax = taxable * tax_rate / 100
        total_tax += tax
        print(name, "Qty:", quantity, "Line: Rs.", round(line, 2),
              "Tax: Rs.", round(tax, 2))

    final_total = subtotal - discount + total_tax
    print("Subtotal: Rs.", round(subtotal, 2))
    print("Discount: Rs.", round(discount, 2))
    print("Tax total: Rs.", round(total_tax, 2))
    print("Amount due: Rs.", round(final_total, 2))
```

8. Test Cases

Scenario	Expected result
Subtotal below ■2,000	No discount
Subtotal ■2,000	10% discount begins
Subtotal ■8,000	Discount capped at ■500
Negative product price	Invalid product record

9. Sample Output

```
ITEMIZED INVOICE
Notebook Qty: 5 Line: Rs. 400 Tax: Rs. 18.0
Backpack Qty: 2 Line: Rs. 1800 Tax: Rs. 194.4
Pen Pack Qty: 3 Line: Rs. 360 Tax: Rs. 16.2
Subtotal: Rs. 2560
Discount: Rs. 256.0
Tax total: Rs. 228.6
Amount due: Rs. 2532.6
```

10. Conclusion

The invoice combines product quantities, category tax rates, a threshold discount, and a discount cap to calculate the amount due.

CASE STUDY 8: Manufacturing Cost and Profitability Analysis

Reference in supplied list: Case Study 8

1. Problem Statement

Calculate manufacturing cost from material, labour, wastage, and machine costs. Determine unit cost, selling revenue, profit, and whether a target profit percentage is achieved.

2. Objectives

• Calculate total production cost from multiple components.

• Account for material wastage and unit selling price.

• Compare profit percentage with a target.

3. Assumptions

• Material cost is the cost of material used before wastage.

• Wastage cost is calculated as a percentage of material cost.

• Total production cost is material plus wastage, labour, and machine cost.

• Profit percentage is profit divided by total production cost multiplied by 100.

4. Algorithm

1. Read units produced, material cost, wastage percentage, labour, machine cost, selling price per unit, and target profit percentage.

2. Validate non-negative inputs and positive production quantity.

3. Calculate wastage cost and total cost.

4. Calculate revenue, profit, unit cost, and profit percentage.

5. Compare profit percentage to the target and display the result.

5. Flowchart

```
START → Read production inputs\n  ↓\nValidate inputs and units > 0\n  ↓\nCalculate wastage cost\n  ↓\nTota
```

6. Pseudocode

```
READ units, material, wastage_rate, labour, machine, selling_price, target\nIF units <= 0 or any cost is negative: show inval
```

7. Python Source Code

```
# Manufacturing Cost and Profitability Analysis
units = 500
material_cost = 20000
wastage_percent = 5
labour_cost = 8000
machine_cost = 4000
selling_price = 75
target_profit_percent = 20

inputs = [material_cost, wastage_percent, labour_cost,
          machine_cost, selling_price, target_profit_percent]

if units <= 0 or min(inputs) < 0:
    print("Invalid production data")
else:
    wastage_cost = material_cost * wastage_percent / 100
    total_cost = material_cost + wastage_cost + labour_cost + machine_cost
    revenue = units * selling_price
    profit = revenue - total_cost
    unit_cost = total_cost / units
    profit_percent = (profit / total_cost) * 100 if total_cost > 0 else 0

    print("Units produced:", units)
```

```
print("Wastage cost: Rs.", round(wastage_cost, 2))
print("Total production cost: Rs.", round(total_cost, 2))
print("Revenue: Rs.", round(revenue, 2))
print("Profit: Rs.", round(profit, 2))
print("Cost per unit: Rs.", round(unit_cost, 2))
print("Profit percentage:", round(profit_percent, 2), "%")
if profit_percent >= target_profit_percent:
    print("Target profit achieved")
else:
    print("Target profit not achieved")
```

8. Test Cases

Scenario	Expected result
Units produced = 500	Unit cost calculated
Selling price above unit cost	Positive profit if revenue exceeds total cost
Target 20%	Compare actual profit percentage
Units = 0	Invalid production data

9. Sample Output

```
Units produced: 500\nWastage cost: Rs. 1000.0\nTotal production cost: Rs. 33000.0\nRevenue: Rs. 37500\nProfit: Rs. 4500.0\nCo
```

10. Conclusion

The program measures production cost and profitability, including wastage, and checks whether the target profit margin is reached.

CASE STUDY 9: Multi-Currency Travel Budget Analysis

Reference in supplied list: Case Study 9

1. Problem Statement

Convert travel expenses in different currencies using supplied exchange rates, add a conversion fee, compare total spending against a budget, and identify the largest expense category.

2. Objectives

• Store expense records in a list.

• Convert each expense to the home currency using given rates.

• Calculate fees, total spending, budget balance, and the largest category.

3. Assumptions

• All exchange rates are supplied sample rates in INR per one unit of foreign currency.

• A 2% conversion fee is added to each converted expense.

• The home currency is Indian Rupees (INR).

• Rates are examples for programming practice and are not live exchange rates.

4. Algorithm

1. Read or define expense category, amount, currency, and exchange rate.

2. Convert each amount to INR.

3. Calculate 2% conversion fee and adjusted expense.

4. Accumulate total spending and identify the largest adjusted category.

5. Compare total spending with the INR budget and report remaining amount or overspend.

5. Flowchart

```
START → Load expenses, rates, and budget\n  ↓\nFor each expense: convert to INR\n  ↓\nAdd 2% conversion fe
```

6. Pseudocode

```
total = 0; largest_amount = -1\nFOR each expense\n    converted = amount * rate\n    fee = converted * 2%\n    adjusted = con
```

7. Python Source Code

```
# Multi-Currency Travel Budget Analysis
# Sample rates only: INR per one unit of foreign currency
expenses = [
    ["Hotel", 120, "USD", 83.0],
    ["Meals", 80, "EUR", 90.0],
    ["Transport", 5000, "JPY", 0.56],
    ["Museum", 40, "USD", 83.0]
]
budget = 20000
fee_rate = 0.02

total = 0
largest_category = ""
largest_amount = -1

for expense in expenses:
    category, amount, currency, rate = expense
    if amount < 0 or rate <= 0:
        print("Invalid expense:", category)
        break
    converted = amount * rate
    fee = converted * fee_rate
    adjusted = converted + fee
    total += adjusted
    print(category, "(" + currency + "): Rs.", round(adjusted, 2))
    if adjusted > largest_amount:
        largest_amount = adjusted
```

```
        largest_category = category
else:
    print("Total spending: Rs.", round(total, 2))
    print("Largest category:", largest_category,
          "- Rs.", round(largest_amount, 2))
    if total <= budget:
        print("Within budget. Balance: Rs.", round(budget - total, 2))
    else:
        print("Over budget by: Rs.", round(total - budget, 2))
```

8. Test Cases

Scenario	Expected result
USD 120 at ■83	Base conversion ■9,960
2% fee	■199.20 on ■9,960 conversion
Total below budget	Show remaining balance
Invalid rate 0	Reject expense record

9. Sample Output

```
Hotel (USD): Rs. 10159.2
Meals (EUR): Rs. 7344.0
Transport (JPY): Rs. 2856.0
Museum (USD): Rs. 3386.4
Total spending: Rs. 23745.6
Largest category: Hotel - Rs. 10159.2
Over budget by: Rs. 3745.6
```

10. Conclusion

The solution converts mixed-currency expenses using supplied rates, adds conversion fees, and identifies both the largest expense and budget overrun.

CASE STUDY 10: Campus Event Financial Planning

Reference in supplied list: Case Study 10

1. Problem Statement

Estimate campus event expenses for venue, equipment, catering, transport, and certificates. Calculate sponsorship shortfall, participant contribution, and the minimum registrations needed to break even.

2. Objectives

• Add fixed and variable event expenses.

• Calculate sponsorship shortfall and required participant contribution.

• Determine the minimum number of registrations needed to cover costs.

3. Assumptions

• Venue, equipment, and transport are fixed costs; catering and certificates are per participant.

• Sponsorship is deducted from total estimated costs.

• Registration fee is a fixed amount per participant.

• Break-even registrations are the ceiling of (fixed costs minus sponsorship) divided by (registration fee minus variable cost per participant), provided each registration contributes a positive amount.

4. Algorithm

1. Read fixed costs, per-person costs, sponsorship, and registration fee.

2. Validate values and ensure registration fee is greater than variable cost per participant.

3. Calculate total fixed cost, variable cost per participant, and net fixed cost after sponsorship.

4. Calculate the minimum registrations required to break even.

5. For the planned registration count, calculate total cost, registration income, sponsorship, and surplus/shortfall.

6. Display financial summary.

5. Flowchart

```
START → Read cost and registration inputs\n  ↓\nValidate non-negative values\n  ↓\nCalculate fixed cost an
```

6. Pseudocode

```
fixed = venue + equipment + transport\nvariable_per_person = catering + certificate\nnet_fixed = max(0, fixed - sponsorship)\n
```

7. Python Source Code

```
# Campus Event Financial Planning
venue = 12000
equipment = 5000
transport = 3000
catering_per_person = 180
certificate_per_person = 20
sponsorship = 10000
registration_fee = 300
planned_participants = 100

values = [venue, equipment, transport, catering_per_person,
          certificate_per_person, sponsorship, registration_fee,
          planned_participants]

if min(values) < 0:
    print("Invalid event data")
else:
    fixed_cost = venue + equipment + transport
    variable_per_person = catering_per_person + certificate_per_person
    contribution = registration_fee - variable_per_person
    net_fixed = max(0, fixed_cost - sponsorship)

    if contribution <= 0:
```

```
        print("Break-even is not possible at this registration fee")
    else:
        minimum_people = (net_fixed + contribution - 1) // contribution
        planned_cost = fixed_cost + planned_participants * variable_per_person
        registration_income = planned_participants * registration_fee
        total_income = registration_income + sponsorship
        balance = total_income - planned_cost

        print("Fixed cost: Rs.", fixed_cost)
        print("Variable cost per participant: Rs.", variable_per_person)
        print("Sponsorship: Rs.", sponsorship)
        print("Minimum break-even registrations:", minimum_people)
        print("Planned participants:", planned_participants)
        print("Estimated total cost: Rs.", planned_cost)
        print("Total income: Rs.", total_income)
        print("Surplus: Rs.", balance)
        if balance < 0:
            print("Shortfall: Rs.", abs(balance))
        else:
            print("Event budget covered")
```

8. Test Cases

Scenario	Expected result
Fixed costs ■20,000; sponsorship ■10,000	Net fixed cost ■10,000
Variable cost ■200; fee ■300	Contribution ■100 per participant
Net fixed ■10,000 and contribution ■100	Break-even at 100 participants
Fee equals variable cost	Break-even not possible

9. Sample Output

```
Fixed cost: Rs. 20000\nVariable cost per participant: Rs. 200\nSponsorship: Rs. 10000\nMinimum break-even registrations: 100\n
```

10. Conclusion

The program estimates event expenses and revenue, calculates the minimum break-even registrations, and reports whether the planned event is financially viable.

Final Submission Checklist

■ Fill in the student name, roll number, branch, section, and faculty details on the cover page.

■ Run every Python program in your own Python environment and capture genuine output screenshots.

■ Check that the test cases match the actual program output.

■ Add any flowchart images or formal diagrams required by your faculty; the flowcharts in this PDF are text-based logic flows.

■ Create your own GitHub repository named Python-Activity-Based-Learning, as specified in the circular.

■ Upload the 10 Python programs, supporting documentation, screenshots, and one consolidated PDF.

■ Add a README.md with student details and the titles of the 10 selected case studies.

■ Submit the repository URL to the Python Programming faculty member / Lab Mentor on the first working day after the holidays.

Academic integrity note

This document is a worked study template. Review the logic, test each program, personalize the work, and be ready to explain every statement independently, as required by the college circular.